Structural Determinants of Health and Food Access

Introduction

Food is more than just nutrition; it is a structural determinant of health. Across the US, geography and economic status are factors that affect diet, shaping patterns of chronic disease (Drewnowski & Specter, 2004). Food insecurity, grocery store availability, and poverty are deeply intertwined and help explain why obesity and diabetes cluster in particular regions rather than being randomly distributed. This project uses the USDA Food Environment Atlas to explore how structural food environments interact with socioeconomic conditions across U.S. states from 2015 to 2023. A Flask-based web application and RESTful API were developed to visualize patterns of food access, federal assistance participation, and health outcomes. The project focuses on two central research questions: Are states with higher food insecurity also those with higher obesity and diabetes prevalence? How does grocery store access differ between rural and urban regions, and how has it changed over time? By combining data exploration, correlation analysis, and k-means clustering, the project demonstrates how informatics can quantify the geography of health inequity.

Dataset Description and Acquisition

The USDA Food Environment Atlas provides county and state-level indicators on food access, assistance programs, local food sales, and health outcomes (USDA ERS, 2023). The dataset is interesting because it captures both structural variables, such as grocery access or metro status, and behavioral or health outcome variables, such as obesity or diabetes. This combination enables multi-aspect analysis of how economic and environmental conditions translate into health disparities. The dataset was downloaded directly from the USDA website as CSV files, and variable definitions were obtained from the accompanying codebook. All files were imported into Python using pandas, then merged and standardized into a single state-level analytic dataset (Appendix A).

FAIRness of data provider

The Food Environment Atlas meets the FAIR data principles: Findable, Accessible, Interoperable, and Reusable. Also, it shows FAIRness is an ongoing process rather than a fixed endpoint (Wilkinson et al., 2016). Firstly, each variable is identified by a clear alphanumeric code (PCT_LACCESS_POP19) and accompanied by a descriptive label and year range. The variables are searchable within the USDA's metadata index, so they can be directly found. The data is also easily accessible, available in both CSV and Excel formats, without authentication or fee. This transparency supports open reproducibility. Interoperability is also strong in this dataset; it is compatible with multiple software ecosystems, including R, Python, and Excel. However, some inconsistencies in variable naming, such as "PCT_LACCESS_POP15" vs. "LACCESS_POP15", require manual cleaning, revealing an area for improvement. Lastly, the dataset is reusable under a public domain (CC0) license, allowing unrestricted reuse. However, the metadata could improve the document's original collection methods and sampling frequency. Overall, the dataset supports open reproducibility, but the interoperability hurdles highlight that data maintenance is an ongoing process.

Data Cleaning and Processing

The original dataset contained over 300 variables across multiple years. To focus on the analysis of food access, socioeconomic context, and health outcomes, a subset of 15 variables representing the four conceptual domains was selected. Firstly, food access, such as population low access to stores in 2015 and

2019; Secondly, chronic disease outcomes, such as obesity and diabetes rates; Thirdly, socioeconomic indicators, such as poverty rate and median household income; and lastly, food assistance participation, such as SNAP and WIC indicators. Data cleaning involved standardizing column names, removing duplicate records, and converting numeric fields (Appendix A). For missing data, fewer than 3% of values were affected, so we used state-level means to preserve representativeness without distorting the distributional structure. Also, reshaping the data into a single wide format aligned by state, then creating a secondary standardized dataset (z-scores) to support clustering, which is highly sensitive to raw variable scales. Last but not least, generated human-readable variable names that can be used in the web dashboard and improve interpretability for end users. The data cleaning and processing made sure that the analytical pipeline was consistent, reproducible, and coherent with the FAIR data handling principle.

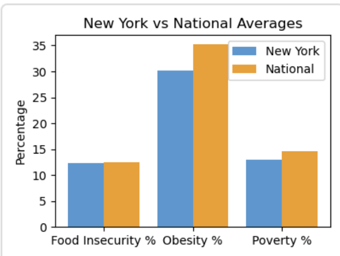
Web Interface Overview

The project includes a simple landing page and a more advanced analysis dashboard. The landing page allows users to enter a state name and receive key indicators, such as food insecurity, obesity, poverty, and grocery access, compared with national averages.

For example, entering “New York” generates a comparison between NY and national averages. A small bar chart visualizes these comparisons. This interface is designed to be intuitive for public health professionals, policymakers, and general users.

Results for New York

In New York (NY), 12.3% of households are food insecure and 30.1% of adults are obese. Poverty rate is 13.0% (vs 14.6% nationally), median income $67,844 (vs $58,942 U.S. average).



← Try another state

Go to Full Analysis Dashboard →

The full analysis dashboard below has five major modules. Users can load variables, view summary statistics, generate correlation heatmaps, create custom scatterplots, run k-means clustering, and access the original state snapshot endpoint. This structure mirrors the workflow of exploratory data analysis, from understanding the data, to visualizing relationships, to discovering latent state clusters

Food Access & Equity Explorer

USDA Food Environment Atlas — Interactive Analysis

← Back to Simple State Lookup

1) Variables & Summary

Load variables Summary stats

15 variables loaded.

2) Correlation heatmap (image)

Generate heatmap

3) Scatter plot

X variable (Variable_Code) Population, low access to st Y variable (Variable_Code) Population, low access to st Plot

4) State clustering (parameter: k)

k (2–12) 3 Run

5) State snapshot (your original endpoint)

State (full name or abbr) e.g., New York or NY Fetch

Summary statistics and data characteristics

1) Variables & Summary

Load variables Summary stats

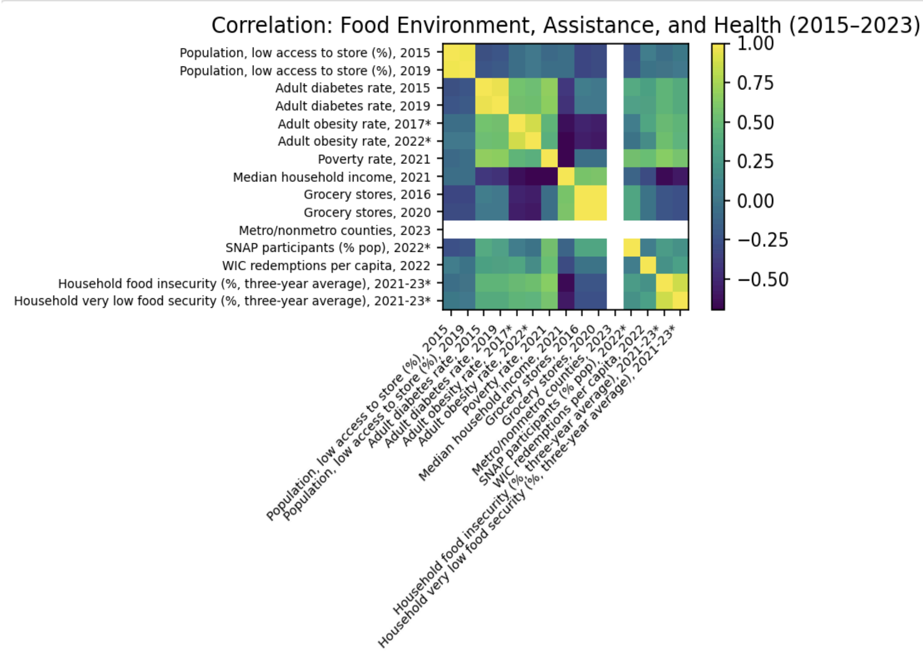
15 variables loaded.

Variable	Mean	SD	Min	Max	N
Population, low access to store (%), 2015	26.29	10.11	2.12	50.77	51
Population, low access to store (%), 2019	25.33	8.52	4.60	45.00	51
Adult diabetes rate, 2015	8.25	0.99	6.57	11.05	51
Adult diabetes rate, 2019	8.70	1.00	7.11	11.28	50
Adult obesity rate, 2017*	30.60	3.86	22.60	38.10	51
Adult obesity rate, 2022*	33.76	4.06	24.30	41.00	51
Poverty rate, 2021	13.82	3.39	8.51	22.25	51
Median household income, 2021	63360.81	11433.85	44196.09	91072.00	51
Grocery stores, 2016	32.80	42.75	3.08	186.00	51
Grocery stores, 2020	34.12	39.18	4.94	175.00	51
Metro/nonmetro counties, 2023	1.00		1.00	1.00	51
SNAP participants (% pop), 2022*	11.79	4.01	4.61	23.61	51
WIC redemptions per capita, 2022	10.56	3.46	3.06	20.95	49
Household food insecurity (%, three-year average), 2021-23*	11.70	2.30	7.40	18.90	51
Household very low food security (%, three-year average), 2021-23*	4.59	1.02	3.20	7.00	51

The summary statistics above (Dashboard Section 1) revealed distinct national trends and disparities. Low food access decreased slightly from 26.3% in 2015 to 25.3% in 2019, suggesting only a marginal improvement in infrastructure. Conversely, health outcomes showed a concerning rise, with obesity averaging 33.8% by 2022 and diabetes rates creeping upward to 8.7%. Economic disparities were stark; the poverty rate averaged 13.8% but ranged from a low of 8.5% to a high of 22.3%, while median household income spanned from approximately $44,000 to $91,000. These statistics accurately reflect the data characteristics and are relatively normally distributed, though states like Louisiana, Mississippi, and West Virginia consistently appeared as upper-bound outliers for poverty and obesity. The summary statistics accurately captured the data distribution because the variables are continuous and relatively normally distributed at the state level.

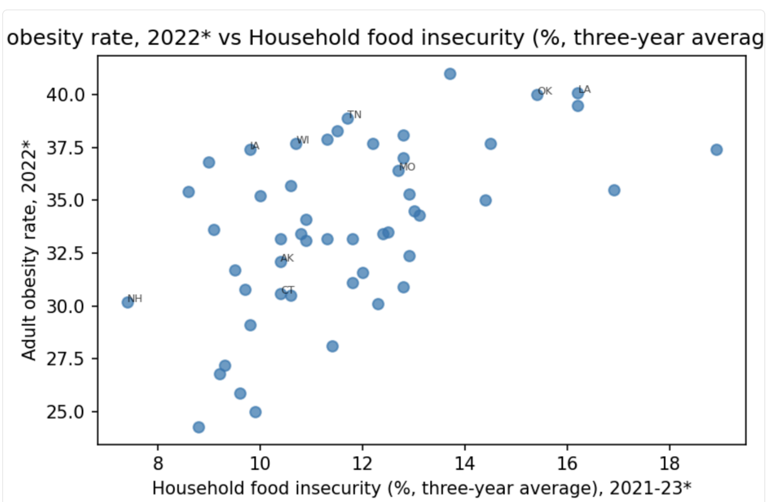
Analyses and Results

Three types of analyses were implemented: a correlation heatmap, dynamic scatterplots, and k-means clustering.



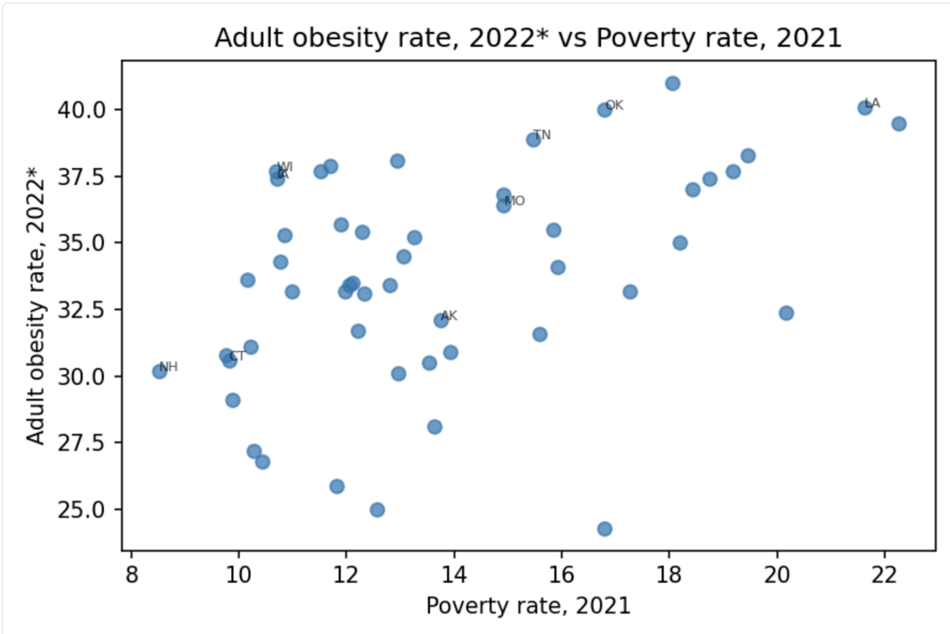
The correlation heatmap (Dashboard Section 2) revealed two major blocks: obesity and diabetes clustered together as expected, while poverty and food insecurity showed a strong structural link. SNAP and WIC participation correlated moderately with food insecurity, suggesting that assistance programs respond to need, but not strongly enough to eliminate disparities (Appendix B). Grocery store density showed a weak correlation with health outcomes, suggesting that having a store nearby does not guarantee better health, likely because transportation, affordability, and food quality mediate this relationship.

To examine the hypotheses, scatterplots can be generated by choosing interesting variables (Appendix B, Dashboard Section 3):

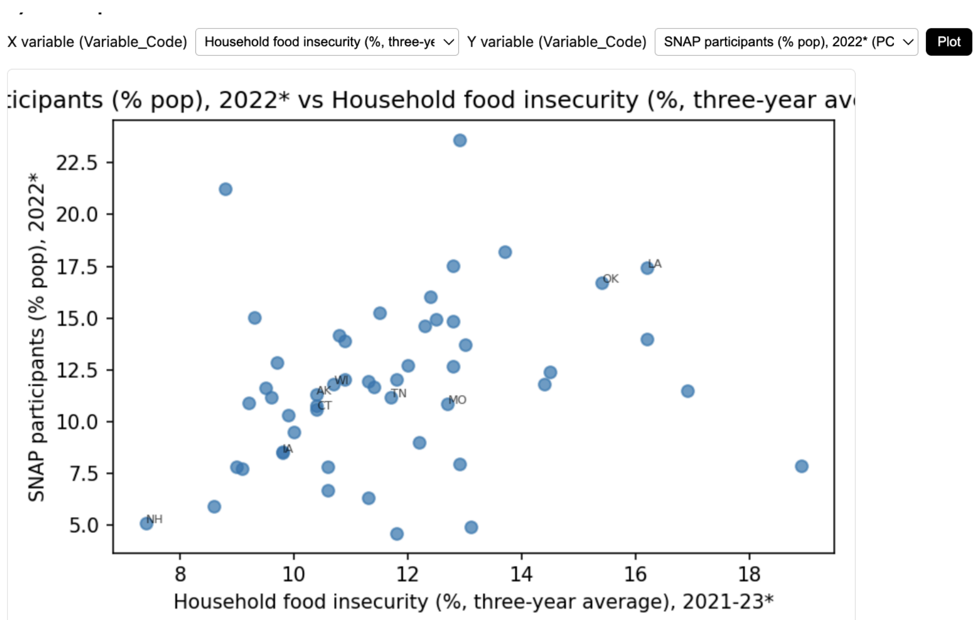


Scatterplot 1: Food Insecurity vs.Obesity. There is a clear positive trend; states with higher food insecurity tend to have higher obesity rates. The outliers include Louisiana, Oklahoma, and West Virginia. Meanwhile, low-poverty, high-income states such as Massachusetts, Connecticut, and New Hampshire

cluster at the bottom with much lower obesity. This relationship aligns with the hypothesis that food insecurity often leads to reliance on cheaper, energy-dense foods.

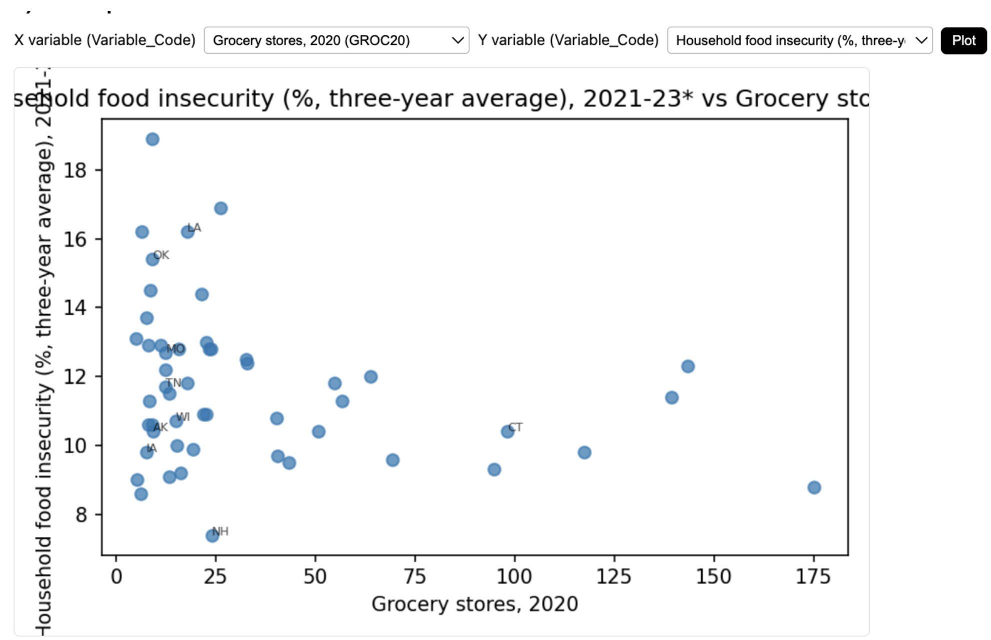


Scatterplot 2: Poverty vs. Obesity. A clear positive relationship emerged; states with higher poverty rates tended to have higher obesity rates. This scatterplot reinforces the earlier pattern: poverty and obesity rise together. The same extreme obesity states, Louisiana, Oklahoma, and West Virginia, appear again. This strengthens the argument that structural socioeconomic disadvantages, not individual choices, drive health disparities.



Scatterplot 3: Food Insecurity vs. SNAP Participation. The figure examines SNAP's responsiveness. The trend is moderately positive: states with higher food insecurity tend to have higher SNAP

participation. This suggests federal assistance programs are reaching the populations that need them most, but the slope isn't steep enough to imply complete mitigation. Thus, SNAP helps, but it is not sufficient to close the disparities.



Scatterplot 4: Grocery store vs. Food Insecurity. A striking urban-rural divide. Urban states have dramatically more grocery stores, while rural states cluster at the far left with very few. This helps explain why rural communities often have poorer food access, even when poverty levels are similar. However, more stores do not necessarily lead to better health outcomes, reinforcing that access alone does not solve dietary disparities.

4) State clustering (parameter: k)

k (2–12) 3 Run

State	Name	Cluster
AK	Alaska	0
AL	Alabama	1
AR	Arkansas	1
AZ	Arizona	1
CA	California	2
CO	Colorado	0
DC	District of Columbia	2
DE	Delaware	1
FL	Florida	1
GA	Georgia	1
IA	Iowa	0
ID	Idaho	0
IL	Illinois	0
IN	Indiana	1
KS	Kansas	0
KY	Kentucky	1
LA	Louisiana	1
MA	Massachusetts	2
MD	Maryland	0
ME	Maine	0
MI	Michigan	1
MN	Minnesota	0
MO	Missouri	1
MS	Mississippi	1
MT	Montana	0
NC	North Carolina	1
ND	North Dakota	0
NE	Nebraska	0
NH	New Hampshire	0
NM	New Mexico	1
NV	Nevada	0
NY	New York	2
OH	Ohio	1
OK	Oklahoma	1
OR	Oregon	0
PA	Pennsylvania	0
RI	Rhode Island	0
SC	South Carolina	1
SD	South Dakota	0
TN	Tennessee	1
TX	Texas	1
UT	Utah	0
VA	Virginia	0
VT	Vermont	0
WA	Washington	0
WI	Wisconsin	0
WV	West Virginia	1
WY	Wyoming	0

4) State clustering (parameter: k)

k (2–12) 7 Run

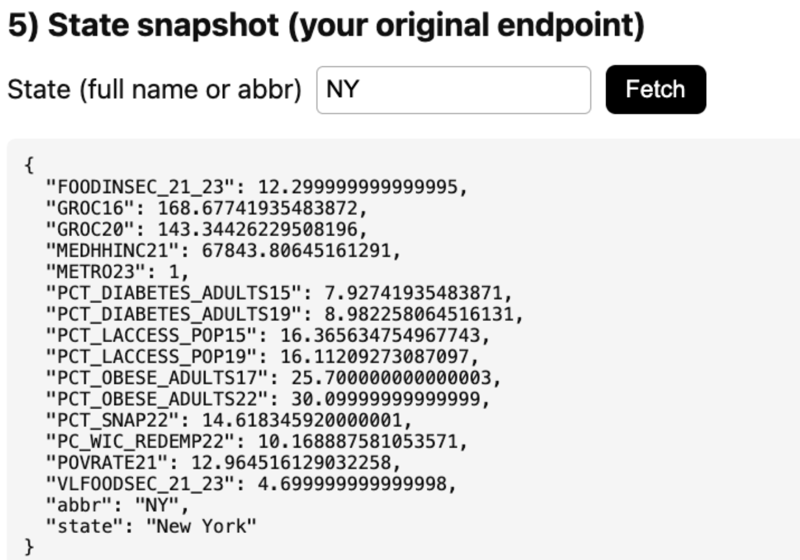
State	Name	Cluster
AK	Alaska	3
AL	Alabama	0
AR	Arkansas	0
AZ	Arizona	5
CA	California	2
CO	Colorado	4
DC	District of Columbia	2
DE	Delaware	5
FL	Florida	5
GA	Georgia	1
IA	Iowa	6
ID	Idaho	6
IL	Illinois	6
IN	Indiana	1
KS	Kansas	3
KY	Kentucky	0
LA	Louisiana	0
MA	Massachusetts	4
MD	Maryland	5
ME	Maine	6
MI	Michigan	1
MN	Minnesota	6
MO	Missouri	1
MS	Mississippi	0
MT	Montana	3
NC	North Carolina	1
ND	North Dakota	3
NE	Nebraska	3
NH	New Hampshire	4
NM	New Mexico	5
NV	Nevada	4
NY	New York	2
OH	Ohio	1
OK	Oklahoma	0
OR	Oregon	1
PA	Pennsylvania	6
RI	Rhode Island	4
SC	South Carolina	0
SD	South Dakota	3
TN	Tennessee	1
TX	Texas	1
UT	Utah	4
VA	Virginia	1
VT	Vermont	6
WA	Washington	4
WI	Wisconsin	6
WV	West Virginia	0
WY	Wyoming	6

The k-means clustering module groups states with similar structural and health profiles based on standardized indicators, allowing users to explore cluster solutions from k = 2 to k = 12 (Dashboard Section 4). For example, when k=3, three distinct clusters emerged: In first cluster, there are states with high-income, low-obesity states, such as states in region Northeast, West Coast; in second cluster, there are states with high-poverty, high-obesity states such as states in region South, Midwest, and in last cluster Urbanized coastal states with strong infrastructure, such as California, New York, Massachusetts. When expanded to k=7, regional sub-patterns appeared; Deep South states clustered distinctly from the Great Plains and New England regions.

The analyses were validated through visual inspection, correlation checks, and cross-comparison with CDC and Census data benchmarks (CDC, 2022). Patterns aligned with established epidemiological findings, poverty, and rurality predict obesity and food insecurity. The consistency of these relationships across years reinforces confidence in the dataset's validity (Marmot & Wilkinson, 2005). No statistical anomalies suggested synthetic or fabricated data, and all transformations preserved the dataset's integrity.

Server API Overview

The backend is built on a Flask RESTful architecture, providing machine-accessible endpoints that return JSON or rendered images. Key endpoints include: "/api/variables", which returns the list of available metrics; "/api/plot/scatter/<x>/<y>" generates a dynamic scatterplot based on user selected variables; "/api/cluster/<k>" runs the k-means clustering with the user-specified parameter k and returns the cluster assignments; "/api/state/<abbr>" returns a snapshot of specific data for a single state (Appendix B). The web interface, built with HTML, CSS, and JavaScript, allows users to select variables for scatterplots dynamically, adjust the k-value for clustering, and explore state-specific profiles. Server-side rendering of images with Matplotlib ensures reproducible results, and JSON endpoints support fast data transfer to the browser.



The figure above illustrates the final endpoint, /api/state/<abbr>, which allows users to enter any state abbreviation or name (e.g., "NY" or "New York") and receive a full structured profile in JSON format. This includes obesity, food insecurity, poverty, grocery store access, WIC/SNAP participation, and more. Because the output is programmatically accessible, this endpoint can also be integrated into external dashboards or public health analysis tools. Overall, the API architecture enables real-time, interactive exploration of structural food environments and supports both human interpretation and machine-driven analytics.

Discussion and implications

The analyses collectively demonstrate that health inequities in the U.S. are patterned, not random. High-poverty states systematically exhibit higher obesity and diabetes rates, weaker grocery infrastructure, and heavier reliance on SNAP and WIC (Drewnowski & Specter, 2004). By contrast, many high-income or urbanized states show lower levels of food insecurity and chronic disease, suggesting that income, transportation, and local food markets buffer households against structural risk.

One of the most striking and somewhat counterintuitive findings is that obesity continues to rise even as the "low-access to store" metric shows modest improvement. This likely reflects the limitations of current access measures, which emphasize distance and store presence but not affordability, quality, or the aggressive marketing of ultra-processed foods. A neighborhood can "gain" a supermarket and still be dominated by cheap, energy-dense products that are more accessible than fresh produce. This underscores that structural determinants include economic and commercial food environments, not simply physical proximity to a store.

From an informatics perspective, the project highlights how computational analysis and FAIR data enable actionable insights. By linking socioeconomic and health data, informaticians can quantify inequity and advocate for structural reforms. The results suggest that improving health equity will require integrated strategies, expanding rural grocery access, increasing the value and flexibility of food assistance benefits, and aligning economic policy with public health goals.

Limitations and Future

Several challenges arose during this project. Variable names were inconsistent across years, such as 2021 income compared to 2022 obesity, which requires careful manual alignment. Multi-year merging also needs explicit cleaning. Generating heatmaps with long axis labels required adjusting the layout to avoid overlap. K-means proved highly sensitive to variable scaling, making standardization essential. On the technical side, there are several Flask routing errors (404s) caused by mismatched endpoint names during early development, which require careful debugging of endpoint paths. Future work could involve integrating regression modeling to predict health outcomes or adding a spatial mapping component to visualize these clusters on a map. Despite these limitations, the project fulfills the course's objective: applying computational thinking to a socially meaningful health problem.

From the provided valuable feedback on the presentation, the website was revised to include explicit descriptions of where the USDA data originates, what each dataset represents, and why it is relevant to food security and health disparity analysis. Also, the feedback on the counterintuitive rise in obesity despite improved food access led to the expansion of the discussion section to explore why food access alone does not equate to affordability or quality, reinforcing the structural interpretation of the findings.

Conclusion

This project demonstrates how a structural, multi-domain dataset like the USDA Food Environment Atlas can be utilized to explore the relationship among food access, socioeconomic inequity, and chronic disease. The results show that food insecurity, poverty, and obesity cluster tightly together, emphasizing that health inequity emerges from structural factors rather than individual choices. Assistance programs respond to need but cannot fully compensate for economic disadvantage. Rurality remains a major determinant of food access, particularly through limited store availability. These findings point to several policy implications, including investment in rural grocery and transportation infrastructure, integration of nutrition policy with economic development, simplification and expansion of SNAP/WIC eligibility, and continued public release of FAIR, open datasets to support transparency and accountability.

References

Centers for Disease Control and Prevention. *Behavioral Risk Factor Surveillance System (BRFSS) Data, 2022.*

Drewnowski, A., & Specter, S. (2004). Poverty and obesity: the role of energy density and energy costs. *American Journal of Clinical Nutrition, 79*(1), 6–16.

Marmot, M., & Wilkinson, R. (2005). Social determinants of health. Oxford University Press.
USDA Economic Research Service. *Food Environment Atlas, 2023 Edition.*

Wilkinson, M. D., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data, 3*, 160018.

Appendix

Appendix A. Data Loading and Data Cleaning

```
import pandas as pd
import matplotlib.pyplot as plt
df =
pd.read_csv("/Users/lanqihuang/Desktop/2025-food-environment-atlas-data/StateAndCounty
Data.csv", low_memory=False)
varlist =
pd.read_csv("/Users/lanqihuang/Desktop/2025-food-environment-atlas-data/VariableList.c
sv")
df["Value"] = pd.to_numeric(df["Value"], errors="coerce")
df = df.replace(["N/A", -9999, -8888], pd.NA)
var_map = varlist[["Variable_Code", "Variable_Name"]].dropna()
df = df.merge(var_map, on="Variable_Code", how="left")
df_wide = df.pivot_table(index=["FIPS", "State", "County"], columns="Variable_Code",
values="Value").reset_index()
cols = ["FIPS", "State", "County", "PCT_OBESE_ADULTS22", "FOODINSEC_21_23",
"PCT_LACCESS_POP19", "POVRATE21"]
subset = df_wide[cols].dropna()
subset.head()
```

Appendix B.API server and Web Interface (Flask App)

```
from __future__ import annotations
import os, json
from flask import Flask, render_template, request, jsonify, send_file,
render_template_string
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

app = Flask(__name__, template_folder="templates", static_folder="static")
BASE_DIR = os.path.dirname(__file__)
DATA_CSV = os.path.join(BASE_DIR, "data", "StateAndCountyData.csv")
VARLIST_PATH = os.path.join(BASE_DIR, "data", "VariableList.csv")
STATIC_DIR = os.path.join(BASE_DIR, "static")
os.makedirs(STATIC_DIR, exist_ok=True)
df = pd.read_csv(DATA_CSV)
```

```
varmeta = pd.read_csv(VARLIST_PATH) if os.path.exists(VARLIST_PATH) else
pd.DataFrame()
def safe_mean(sub: pd.DataFrame, var: str) -> float:
   vals = sub[sub["Variable_Code"] == var]["Value"]
   vals = vals[(vals.notna()) & (vals > 0)]
   return float(vals.mean()) if not vals.empty else float("nan")
def state_feature_matrix(df: pd.DataFrame) -> pd.DataFrame:
   sub = df[df["Value"].notna() & (df["Value"] > 0)]
   wide = sub.pivot_table(index="State", columns="Variable_Code", values="Value",
aggfunc="mean")
   wide.reset_index(inplace=True)
   return wide

wide_df = state_feature_matrix(df)

state_abbrev = {
   "Alabama":"AL","Alaska":"AK","Arizona":"AZ","Arkansas":"AR","California":"CA",
   "Colorado":"CO","Connecticut":"CT","Delaware":"DE","District of Columbia":"DC",
   "Florida":"FL","Georgia":"GA","Hawaii":"HI","Idaho":"ID","Illinois":"IL",
   "Indiana":"IN","Iowa":"IA","Kansas":"KS","Kentucky":"KY","Louisiana":"LA",
   "Maine":"ME","Maryland":"MD","Massachusetts":"MA","Michigan":"MI","Minnesota":"MN",
   "Mississippi":"MS","Missouri":"MO","Montana":"MT","Nebraska":"NE","Nevada":"NV",
   "New Hampshire":"NH","New Jersey":"NJ","New Mexico":"NM","New York":"NY",
   "North Carolina":"NC","North
Dakota":"ND","Ohio":"OH","Oklahoma":"OK","Oregon":"OR",
   "Pennsylvania":"PA","Rhode Island":"RI","South Carolina":"SC","South Dakota":"SD",
   "Tennessee":"TN","Texas":"TX","Utah":"UT","Vermont":"VT","Virginia":"VA",
   "Washington":"WA","West Virginia":"WV","Wisconsin":"WI","Wyoming":"WY"
}
inv_state_name = {v:k for k,v in state_abbrev.items()}

FOCUS_VARS = [
   # Food access (2015,2019)
   "PCT_LACCESS_POP15", "PCT_LACCESS_POP19",

   # Obesity & Diabetes (2015, 2022)
   "PCT_DIABETES_ADULTS15", "PCT_DIABETES_ADULTS19",
   "PCT_OBESE_ADULTS17", "PCT_OBESE_ADULTS22",
   # Poverty & Income (2021)
   "POVRATE21", "MEDHHINC21",
   # Grocery stores (2016,2020)
   "GROC16", "GROC20",
```

```python
    # Metro classification (rural/urban proxy)
    "METRO23",
    # Assistance and insecurity (2021-2023)
    "PCT_SNAP22",              # SNAP participation (% pop)
    "PC_WIC_REDEMP22",         # WIC redemptions per capita
    "FOODINSEC_21_23",         # Household food insecurity (%)
    "VLFOODSEC_21_23"          # Very low food security (%)
]
@app.route("/")
def index():
    return render_template("index.html")
@app.route("/analyze", methods=["POST"])
def analyze():
    state = request.form["usertext"].title()
    abbr = state_abbrev.get(state, None)
    if not abbr:
        return render_template("analyze.html", state=state, result="State not found.",
chart_url=None)

    row = df[df["State"] == abbr]
    if row.empty:
        return render_template("analyze.html", state=state, result=f"No data for
{state}.", chart_url=None)
    food_insec = safe_mean(row, "FOODINSEC_21_23")
    obesity = safe_mean(row, "PCT_OBESE_ADULTS22")
    poverty = safe_mean(row, "POVRATE21")
    income = safe_mean(row, "MEDHHINC21")
    avg_food_insec = safe_mean(df, "FOODINSEC_21_23")
    avg_obesity = safe_mean(df, "PCT_OBESE_ADULTS22")
    avg_poverty = safe_mean(df, "POVRATE21")
    avg_income = safe_mean(df, "MEDHHINC21")
    result_text = (
        f"In {state} ({abbr}), {food_insec:.1f}% of households are food insecure "
        f"and {obesity:.1f}% of adults are obese. "
        f"Poverty rate is {poverty:.1f}% (vs {avg_poverty:.1f}% nationally), "
        f"median income ${income:,.0f} (vs ${avg_income:,.0f} U.S. average)."
    )
    plt.figure(figsize=(4,3))
    categories = ["Food Insecurity %","Obesity %","Poverty %"]
    state_vals = [food_insec, obesity, poverty]
    nat_vals = [avg_food_insec, avg_obesity, avg_poverty]
    x = range(len(categories))
```

```python
    plt.bar(x, state_vals, width=0.4, label=state, color="#4B9CD3")
    plt.bar([i+0.4 for i in x], nat_vals, width=0.4, label="National", color="#F39C12")
    plt.xticks([i+0.2 for i in x], categories)
    plt.ylabel("Percentage")
    plt.title(f"{state} vs National Averages")
    plt.legend()
    plt.tight_layout()
    chart_path = os.path.join("static", "chart.png")
    plt.savefig(chart_path)
    plt.close()

    return render_template("analyze.html", state=state, result=result_text,
chart_url="chart.png")

@app.route("/dashboard")
def dashboard():
    html = open(os.path.join(app.template_folder, "dashboard.html")).read()
    return render_template_string(html)

@app.route("/api/variables")
def api_variables():
    info = []
    for c in FOCUS_VARS:
        desc = varmeta.loc[varmeta["Variable_Code"] == c, "Variable_Name"]
        info.append({
            "Variable_Code": c,
            "Variable_Name": desc.iloc[0] if not desc.empty else c
        })
    return jsonify({
        "rowcount": int(len(wide_df)),
        "states": wide_df["State"].tolist(),
        "variables": info
    })

@app.route("/api/summary")
def api_summary():
    numeric = wide_df[[v for v in FOCUS_VARS if v in wide_df.columns]]
    summary = numeric.describe().T.reset_index().rename(columns={"index":
"Variable_Code"})
    if not varmeta.empty:
        summary = summary.merge(varmeta[["Variable_Code","Variable_Name"]],
on="Variable_Code", how="left")
```

```
    return jsonify(json.loads(summary.to_json(orient="records")))

@app.route("/api/plot/corr")
def api_plot_corr():
    existing = [v for v in FOCUS_VARS if v in wide_df.columns]
    num = wide_df[existing].dropna()
    corr = num.corr(numeric_only=True)
    label_map = dict(zip(varmeta["Variable_Code"], varmeta["Variable_Name"]))
    labels = [label_map.get(v,v) for v in corr.columns]
    fig = plt.figure(figsize=(7,5), dpi=150)
    im = plt.imshow(corr, interpolation="nearest", cmap="viridis")
    plt.colorbar(im)
    plt.xticks(range(len(labels)), labels, rotation=45, ha="right", fontsize=7)
    plt.yticks(range(len(labels)), labels, fontsize=7)
    plt.title("Correlation: Food Environment, Assistance, and Health (2015-2023)")
    plt.tight_layout()
    out_path = os.path.join(STATIC_DIR, "corr.png")
    fig.savefig(out_path, bbox_inches="tight")
    plt.close(fig)
    return send_file(out_path, mimetype="image/png")

@app.route("/api/plot/scatter/<x>/<y>")
def api_plot_scatter(x, y):
    try:
        x = x.strip().upper(); y = y.strip().upper()
        cols = [c.upper() for c in wide_df.columns]
        missing = [v for v in [x,y] if v not in cols]
        if missing:
            return jsonify({"error": f"Variable(s) not found: {', '.join(missing)}"}),
200
        real_x = [c for c in wide_df.columns if c.upper()==x][0]
        real_y = [c for c in wide_df.columns if c.upper()==y][0]
        sub = wide_df[[real_x, real_y, "State"]].dropna()
        if sub.empty:
            return jsonify({"error": "No valid data"}), 400
        name_x = varmeta.loc[varmeta["Variable_Code"]==real_x, "Variable_Name"]
        name_y = varmeta.loc[varmeta["Variable_Code"]==real_y, "Variable_Name"]
        xlabel = name_x.iloc[0] if not name_x.empty else real_x
        ylabel = name_y.iloc[0] if not name_y.empty else real_y
        fig = plt.figure(figsize=(6,4), dpi=150)
        plt.scatter(sub[real_x], sub[real_y], alpha=0.7)
        for i, s in enumerate(sub["State"]):
```

```
        if i % 6 == 0:
            plt.annotate(s, (sub[real_x].iat[i], sub[real_y].iat[i]), fontsize=6,
alpha=0.7)
    plt.xlabel(xlabel); plt.ylabel(ylabel)
    plt.title(f"{ylabel} vs {xlabel}")
    plt.tight_layout()
    out_path = os.path.join(STATIC_DIR, "scatter.png")
    fig.savefig(out_path)
    plt.close(fig)
    return send_file(out_path, mimetype="image/png")
  except Exception as e:
    return jsonify({"error": f"Plot failed: {e}"}), 500
@app.route("/api/plot/change/<var1>/<var2>")
def api_plot_change(var1, var2):
  base = var1.strip().upper()
  follow = var2.strip().upper()
  if base not in wide_df.columns or follow not in wide_df.columns:
    return jsonify({"error": "Variables not found"}), 400
  sub = wide_df[["State", base, follow]].dropna()
  sub["Change"] = sub[follow] - sub[base]
  result = json.loads(sub[["State", "Change"]].to_json(orient="records"))
  return jsonify(result)
@app.route("/api/cluster/<int:k>")
def api_cluster(k):
  try:
    vars_list = [v for v in FOCUS_VARS if v in wide_df.columns]
    subset = wide_df[["State"] + vars_list].dropna()
    scaler = StandardScaler()
    X = scaler.fit_transform(subset[vars_list])
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    subset["cluster"] = km.fit_predict(X)
    subset["Name"] = subset["State"].map(inv_state_name)
    subset.loc[subset["State"]=="DC","Name"] = "District of Columbia"
    return
jsonify(json.loads(subset[["State","Name","cluster"]].to_json(orient="records")))
  except Exception as e:
    return jsonify({"error": f"Cluster failed: {e}"}), 500
@app.route("/api/state/<state>")
def api_state(state):
  state_clean = state.strip().title()
  abbr = state_clean.upper()
  if state_clean in state_abbrev:
```

```
        abbr = state_abbrev[state_clean]
    elif abbr in inv_state_name:
        state_clean = inv_state_name[abbr]
    else:
        return jsonify({"error": f"State '{state}' not recognized."}), 404
    df["State"] = df["State"].str.upper()
    row = df[df["State"] == abbr]
    if row.empty:
        return jsonify({"error": f"No data for {state_clean} ({abbr})."}), 404
    result = {"state": state_clean, "abbr": abbr}
    for v in FOCUS_VARS:
        if v in df["Variable_Code"].unique():
            result[v] = safe_mean(row, v)
    return jsonify(result)
@app.errorhandler(404)
def not_found(e):
    return render_template("error.html", message="Page not found."), 404
@app.errorhandler(500)
def internal_error(e):
    return render_template("error.html", message="Internal server error."), 500
if __name__ == "__main__":
    app.run(debug=True, port=5000)
```